

1. AI 활용 과정 부분 샘플

1) 공연장 좌석 선택 UI(구현)

- 활용 리소스: Claude 3, 구글 검색
- 문제 상황: 10x10 배열의 좌석을 화면에 보여주고, 이미 예매된 좌석(회색), 선택 가능한 좌석(녹색), 현재 선택한 좌석(파란색)으로 상태를 다르게 표시하는 화면을 구현해야 했음
- 초기 Prompt/검색어: *"안드로이드에서 영화관 좌석 선택 화면을 만들고 싶어. GridLayout에 Button 100개를 동적으로 생성해서 넣는 코드를 짜줘."*
(prompt는 기울기체로 표기할 것)
- 해결 과정 및 수정 내역
 - 처음 AI가 제시한 GridLayout 방식은 코드가 직관적이었으나, 좌석 수가 많아질 경우 메모리 성능 저하가 우려되었음
 - 이에 구글링을 통해 대량의 리스트를 보여줄 때는 RecyclerView가 적합하다는 것을 확인함
 - AI에게 다시 *"GridLayout 대신 RecyclerView와 GridLayoutManager를 사용하는 방식으로 코드를 변경하고, 좌석 상태를 Enum 클래스로 관리하도록 구조를 다듬어줘"* 라고 추가 요청함
 - AI가 제공한 어댑터(Adapter) 코드를 프로젝트의 변수명에 맞게 수정하고, 클릭 이벤트 리스너 부분은 수업 시간에 배운 방식을 응용하여 직접 구현함
- 최종 주요 코드
 - 캡처본(주요 로직 부분에 설명 주석 포함)
- 이해 및 성찰
 - 성능을 고려하여 RecyclerView 의 뷰 홀더(ViewHolder) 패턴이 어떻게 뷰를 재사용하여 메모리를 아끼는지 원리를 명확히 이해하게 되었음

2) 동시 예매 처리 로직(디버깅)

- 활용 리소스: StackOverflow 검색, ChatGPT
- 문제 상황: 완성 후 테스트하는 중 두 명의 사용자가 거의 동시에 같은 공연의 동일한 좌석을 클릭하여 예매를 시도할 때, DB에 중복으로 예약 데이터가 들어가는(초과 예매) 심각한 버그를 발견함
- 초기 Prompt/검색어: *"MySQL concurrent ticket booking duplicate problem", "두 명이 동시에 같은 좌석 예매할 때 막는 방법"*
- 해결 과정 및 수정 내역
 - 검색을 통해 '동시성(Concurrency) 문제'라는 것을 알게 됨
 - ChatGPT에게 *"동시성 문제를 해결하기 위해 데이터베이스의 트랜잭션"*

(Transaction)과 락(Lock)을 어떻게 사용해야 하는지 예제 쿼리와 함께 설명해줘'라고 질문하여 개념을 파악함

- 단순한 INSERT 문으로는 한계가 있음을 파악하고, AI의 설명을 바탕으로 SELECT ... FOR UPDATE 구문을 활용한 배타적 락(Exclusive Lock)을 서버 API 로직에 적용하여 중복 예매를 차단함

- 최종 주요 코드

- 캡처본(주요 로직 부분에 설명 주석 포함)

- 이해 및 성찰

- 단순히 코드를 복사하는 것을 넘어 DB의 격리 수준(Isolation Level)과 트랜잭션 처리의 중요성을 배울 수 있었음
 - 앱 클라이언트뿐만 아니라 서버와 DB 간의 안전한 처리 로직이 필수적임을 이해하였음

2. 가장 어려웠던 버그 해결 및 디버깅 과정 샘플

1) UI 업데이트 버그 해결

- 활용 리소스: ChatGPT, Android Developers 공식 문서
- 문제 상황: 공연 목록 화면(Fragment)에서 서버(DB)로부터 대량의 포스터 이미지와 공연 정보를 불러오는 도중, 사용자가 성급하게 '뒤로 가기'를 누르거나 다른 탭으로 이동하면 앱이 강제 종료(Crash)되는 치명적인 현상이 발생함
- 초기 Prompt/에러 로그 입력: *"안드로이드 공연 목록 화면에서 데이터를 서버에서 받아 오는 중에 뒤로 가기를 누르면 앱이 튕겨. Logcat 에러 메시지를 이거야: java.lang.NullPointerException: Attempt to invoke virtual method ... on a null object reference 그리고 IllegalStateException: Fragment not attached to an activity . 원인이 뭘까?"*
- AI의 분석 및 디버깅 과정
 - ① 원인 파악: AI는 에러 로그를 분석한 후, "서버에서 데이터를 다 받아 오기 전에 사용자가 화면을 벗어나서 Fragment가 종료(Destroy)되었는데, 백그라운드 작업이 끝난 뒤 종료된 화면의 UI(Text View, Image View)를 업데이트하려고 시도했기 때문"이라고 정확한 원인을 짚어줌
 - ② 1차 해결: AI가 제안한 대로 네트워크 응답 처리 부분에 if (isAdded() && getActivity() != null) 조건문을 추가하여 화면이 살아있을 때만 UI를 업데이트하도록 수정함. 강제 종료 버그는 해결됨

③ 심화 질문과 코드 개선

- 1차 해결로 작동은 되지만 코드가 지저분해 보여서 AI에게 추가로 질문함
- "이런 생명주기 에러를 막으려고 매번 if문으로 체크하는 건 비효율적인 것 같아. 안드로이드 권장 아키텍처(ViewModel이나 Coroutine)를 써서 생명주기에 맞게 안전하게 처리하는 방법으로 코드를 리팩토링해 줄래?"

④ 최종 해결

- AI가 ViewModel과 viewModelScope를 활용하는 코드를 제시해 줌
- 사용자가 화면을 벗어나면 뷰모델이 클리어되면서 진행 중이던 네트워크 데이터 관찰(Observable)이 안전하게 중단되도록 구조를 완전히 개편함

• 최종 주요 코드

- 캡처본(주요 로직 부분에 설명 주석 포함)

- 이해 및 성찰: Fragment가 Destroy된 후 UI를 업데이트하는 경우 발생하는 에러를 ViewModel로 해결하는 로직을 이해함

2) 서버 응답에 따른 UI 업데이트 버그 해결

- 활용 리소스: ChatGPT, Claude
- 문제 상황
 - 공연장 좌석 정보를 웹 서버(DBMS)에서 불러와 화면(Fragment)에 띄우는 과정에서, 간헐적으로 앱이 강제 종료(Crash)되는 치명적인 버그가 발생함
 - Logcat 확인 결과 NullPointerException이 원인이었으나, 문법적인 오류를 발견하지 못하여 원인을 찾을 수 없었음
- 초기 Prompt/디버깅:
 - 디버깅을 위해 Logcat의 에러 스택 트레이스(Stack Trace) 전체와 데이터를 받아오는 네트워크 통신 코드를 복사하여 AI에게 질문함
 - "안드로이드 앱에서 서버 통신 후 좌석 데이터를 RecyclerView에 연결하는데 간헐적으로 NullPointerException이 발생하면서 앱이 죽어. 에러 로그를 보면 binding.recyclerView 부분에서 터지는데, 원인과 해결 방법을 찾아줘."
- 해결 과정
 - AI의 분석 결과를 통해 '비동기 처리와 안드로이드 생명주기(Lifecycle)'의 충돌 때문임을 파악함. 서버에 데이터를 요청한 후 응답이 오기 전에

사용자가 뒤로가기 버튼 등을 눌러 화면을 벗어나면, 화면(View)이 이미 destroy되었는데도 도착한 데이터를 화면에 그리려고 시도하다가 앱이 죽는 것이었음

- AI가 제안한 해결책을 바탕으로 코드를 전면 수정함. 서버 응답이 도착한 콜백(Callback) 내부에서 데이터를 UI에 적용하기 전에 if (isAdded() && getActivity() != null) 조건문을 추가하여 현재 프래그먼트가 화면에 정상적으로 붙어있는지 먼저 검증하도록 방어 로직을 구현함
- 또한, 네트워크 응답은 백그라운드 스레드에서 처리되므로 UI 업데이트는 `requireActivity().runOnUiThread(...)` 를 사용하여 메인 스레드에서 안전하게 실행되도록 구조를 개선함

- **최종 주요 코드**

- 캡처본(주요 로직 부분에 설명 주석 포함)

- 이해 및 성찰: 모바일 환경에서는 '시간차(비동기)'와 '화면의 상태(생명주기)'를 항상 고려해야 한다는 점을 배웠음